

HDOC – Experiment 34

Question

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D arrays and verify the left distributive property: $A*(B+C) = (A*B) + (A*C)$. The dimensions of input matrices are given as follows: A (m x n), B (n x p), C (n x p).
2. Read three variable-length 2D arrays and verify the right distributive property: $(A+B)*C = (A*C) + (B*C)$. The dimensions of input matrices are given as follows: A (m x n), B (m x n), C (n x p)
3. Exit

❖ Algorithm

Step 1: Start.

Step 2: Display the menu:

1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit

Step 3: Read the user's choice.

Step 4: Use a switch statement.

For **Choice 1 — Left distributive property:**

- Read positive values m, n, and p.
- Declare VLAs:
 - $A[m][n]$
 - $B[n][p]$

- $C[n][p]$
- Read matrices A, B, and C.
- Calculate $B + C$.
- Calculate the LHS: $[A(B+C)]$
- Calculate AB and AC.
- Calculate the RHS: $[AB+AC]$
- Display LHS and RHS matrices.
- Compare corresponding elements.
- If all elements are equal, display that the left distributive property is verified.

For **Choice 2 — Right distributive property:**

- Read positive values m, n, and p.
- Declare VLAs:
 - $A[m][n]$
 - $B[m][n]$
 - $C[n][p]$
- Read matrices A, B, and C.
- Calculate $A + B$.
- Calculate the LHS: $[(A+B) C]$
- Calculate AC and BC.
- Calculate the RHS: $[AC+BC]$
- Display LHS and RHS matrices.
- Compare corresponding elements.
- If all elements are equal, display that the right distributive property is verified.

For **Choice 3:** Exit the program.

For any other choice, display an appropriate error message.

Step 5: Repeat the menu until the user chooses Exit.

Step 6: Stop.

❖ Program code in C

```
#include <stdio.h>
```

```
/* Function prototypes */
```

```
void leftDistributive(void);
```

```
void rightDistributive(void);
```

```
int main(void)
```

```
{
```

```
    int choice;
```

```
    do
```

```
    {
```

```
        printf("\n===== MATRIX DISTRIBUTIVE PROPERTIES =====\n");
```

```
        printf("1. Verify Left Distributive Property\n");
```

```
        printf("   $A*(B+C) = (A*B) + (A*C)$ \n");
```

```
        printf("2. Verify Right Distributive Property\n");
```

```
        printf("   $(A+B)*C = (A*C) + (B*C)$ \n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice)
```

```
        {
```

```
            case 1:
```

```
                leftDistributive();
```

```
        break;

    case 2:
        rightDistributive();
        break;

    case 3:
        printf("Exiting the program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1, 2, or 3.\n");
    }

} while (choice != 3);

return 0;
}
```

```
/* Function to verify left distributive property */
void leftDistributive(void)
{
    int m, n, p;
    int i, j, k;
    int verified = 1;

    printf("\n--- Left Distributive Property ---\n");
```

```

printf("A*(B+C) = (A*B) + (A*C)\n");

/* Read and validate dimensions */
do
{
    printf("Enter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    if (m <= 0 || n <= 0 || p <= 0)
        printf("Dimensions must be positive. Enter again.\n");

} while (m <= 0 || n <= 0 || p <= 0);

/* Variable-length arrays */
int A[m][n];
int B[n][p];
int C[n][p];

int BplusC[n][p];

int LHS[m][p];
int AB[m][p];
int AC[m][p];
int RHS[m][p];

/* Read matrix A */
printf("\nEnter elements of A (%d x %d):\n", m, n);

```

```
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        scanf("%d", &A[i][j]);  
    }  
}
```

```
/* Read matrix B */  
printf("\nEnter elements of B (%d x %d):\n", n, p);
```

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        scanf("%d", &B[i][j]);  
    }  
}
```

```
/* Read matrix C */  
printf("\nEnter elements of C (%d x %d):\n", n, p);
```

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        scanf("%d", &C[i][j]);  
    }  
}
```

```
}
```

```
/* Calculate B + C */
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    for (j = 0; j < p; j++)
```

```
    {
```

```
        BplusC[i][j] = B[i][j] + C[i][j];
```

```
    }
```

```
}
```

```
/* Calculate LHS = A * (B + C) */
```

```
for (i = 0; i < m; i++)
```

```
{
```

```
    for (j = 0; j < p; j++)
```

```
    {
```

```
        LHS[i][j] = 0;
```

```
        for (k = 0; k < n; k++)
```

```
        {
```

```
            LHS[i][j] += A[i][k] * BplusC[k][j];
```

```
        }
```

```
    }
```

```
}
```

```
/* Calculate A * B */
```

```
for (i = 0; i < m; i++)
```

```
{
```

```

for (j = 0; j < p; j++)
{
    AB[i][j] = 0;

    for (k = 0; k < n; k++)
    {
        AB[i][j] += A[i][k] * B[k][j];
    }
}

```

```

/* Calculate A * C */
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        AC[i][j] = 0;

        for (k = 0; k < n; k++)
        {
            AC[i][j] += A[i][k] * C[k][j];
        }
    }
}

```

```

/* Calculate RHS = (A * B) + (A * C) */
for (i = 0; i < m; i++)
{

```

```
    for (j = 0; j < p; j++)  
    {  
        RHS[i][j] = AB[i][j] + AC[i][j];  
    }  
}
```

```
/* Display LHS */
```

```
printf("\nLHS = A*(B+C):\n");
```

```
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        printf("%d\t", LHS[i][j]);  
    }  
    printf("\n");  
}
```

```
/* Display RHS */
```

```
printf("\nRHS = (A*B) + (A*C):\n");
```

```
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        printf("%d\t", RHS[i][j]);  
    }  
    printf("\n");  
}
```

```

}

/* Compare LHS and RHS */
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        if (LHS[i][j] != RHS[i][j])
        {
            verified = 0;
        }
    }
}

if (verified)
{
    printf("\nLeft distributive property is VERIFIED.\n");
    printf("A*(B+C) = (A*B) + (A*C)\n");
}
else
{
    printf("\nLeft distributive property is NOT verified.\n");
}
}

```

```

/* Function to verify right distributive property */
void rightDistributive(void)

```

```

{
    int m, n, p;

    int i, j, k;

    int verified = 1;


    printf("\n--- Right Distributive Property ---\n");
    printf("(A+B)*C = (A*C) + (B*C)\n");


    /* Read and validate dimensions */
    do
    {
        printf("Enter m, n and p: ");
        scanf("%d %d %d", &m, &n, &p);

        if (m <= 0 || n <= 0 || p <= 0)
            printf("Dimensions must be positive. Enter again.\n");

    } while (m <= 0 || n <= 0 || p <= 0);


    /* Variable-length arrays */
    int A[m][n];
    int B[m][n];
    int C[n][p];


    int AplusB[m][n];


    int LHS[m][p];
    int AC[m][p];

```

```
int BC[m][p];

int RHS[m][p];


/* Read matrix A */
printf("\nEnter elements of A (%d x %d):\n", m, n);


for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &A[i][j]);
    }
}


/* Read matrix B */
printf("\nEnter elements of B (%d x %d):\n", m, n);


for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &B[i][j]);
    }
}


/* Read matrix C */
printf("\nEnter elements of C (%d x %d):\n", n, p);
```

```
for (i = 0; i < n; i++)
{
    for (j = 0; j < p; j++)
    {
        scanf("%d", &C[i][j]);
    }
}
```

```
/* Calculate A + B */
for (i = 0; i < m; i++)
{
    for (j = 0; j < n; j++)
    {
        AplusB[i][j] = A[i][j] + B[i][j];
    }
}
```

```
/* Calculate LHS = (A + B) * C */
for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        LHS[i][j] = 0;

        for (k = 0; k < n; k++)
        {
            LHS[i][j] += AplusB[i][k] * C[k][j];
        }
    }
}
```

```
    }  
}
```

```
/* Calculate A * C */  
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        AC[i][j] = 0;  
  
        for (k = 0; k < n; k++)  
        {  
            AC[i][j] += A[i][k] * C[k][j];  
        }  
    }  
}
```

```
/* Calculate B * C */  
for (i = 0; i < m; i++)  
{  
    for (j = 0; j < p; j++)  
    {  
        BC[i][j] = 0;  
  
        for (k = 0; k < n; k++)  
        {  
            BC[i][j] += B[i][k] * C[k][j];  
        }  
    }  
}
```

```
    }  
}
```

```
/* Calculate RHS = (A * C) + (B * C) */
```

```
for (i = 0; i < m; i++)
```

```
{
```

```
    for (j = 0; j < p; j++)
```

```
    {
```

```
        RHS[i][j] = AC[i][j] + BC[i][j];
```

```
    }
```

```
}
```

```
/* Display LHS */
```

```
printf("\nLHS = (A+B)*C:\n");
```

```
for (i = 0; i < m; i++)
```

```
{
```

```
    for (j = 0; j < p; j++)
```

```
    {
```

```
        printf("%d\t", LHS[i][j]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
/* Display RHS */
```

```
printf("\nRHS = (A*C) + (B*C):\n");
```

```
for (i = 0; i < m; i++)
```

```

{
    for (j = 0; j < p; j++)
    {
        printf("%d\t", RHS[i][j]);
    }
    printf("\n");
}

```

```

/* Compare LHS and RHS */

```

```

for (i = 0; i < m; i++)
{
    for (j = 0; j < p; j++)
    {
        if (LHS[i][j] != RHS[i][j])
        {
            verified = 0;
        }
    }
}

```

```

if (verified)

```

```

{
    printf("\nRight distributive property is VERIFIED.\n");
    printf("(A+B)*C = (A*C) + (B*C)\n");
}

```

```

else

```

```

{
    printf("\nRight distributive property is NOT verified.\n");
}

```

}

}

❖ Test case table

Test	Menu Choice	Dimensions	Input Type	Expected Result
1	1	m=2, n=2, p=2	Positive matrices	Left distributive property verified
2	1	m=1, n=1, p=1	Boundary dimension	Left distributive property verified
3	1	m=2, n=3, p=2	Rectangular matrices	Left distributive property verified
4	2	m=2, n=2, p=2	Positive matrices	Right distributive property verified
5	2	m=2, n=3, p=1	Rectangular matrices	Right distributive property verified
6	1 or 2	m=0, n=2, p=2	Invalid dimension	Error; dimensions requested again
7	1 or 2	m=-2, n=2, p=3	Invalid dimension	Error; dimensions requested again
8	4	—	Invalid menu choice	Error message displayed
9	3	—	Exit	Program terminates

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp34]
$ vi array7.c

(kali㉿kali)-[~/Desktop/exp34]
$ gcc array7.c -o array7

(kali㉿kali)-[~/Desktop/exp34]
$ ./array7

===== MATRIX DISTRIBUTIVE PROPERTIES =====
1. Verify Left Distributive Property
    $A*(B+C) = (A*B) + (A*C)$ 
2. Verify Right Distributive Property
    $(A+B)*C = (A*C) + (B*C)$ 
3. Exit
Enter your choice: 1

— Left Distributive Property —
 $A*(B+C) = (A*B) + (A*C)$ 
Enter m, n and p: 2 2 2

Enter elements of A (2 x 2):
1 2
3 4

Enter elements of B (2 x 2):
4 3
2 1

Enter elements of C (2 x 2):
5 6
7 8

LHS =  $A*(B+C)$ :
27 27
342 342

RHS =  $(A*B) + (A*C)$ :
27 27
```

LHS = $A*(B+C)$:

27 27

342 342

RHS = $(A*B) + (A*C)$:

27 27

342 342

Left distributive property is VERIFIED.

$A*(B+C) = (A*B) + (A*C)$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property

$A*(B+C) = (A*B) + (A*C)$

2. Verify Right Distributive Property

$(A+B)*C = (A*C) + (B*C)$

3. Exit

Enter your choice: 1

— Left Distributive Property —

$A*(B+C) = (A*B) + (A*C)$

Enter m, n and p: 1 1 1

Enter elements of A (1 x 1):

2

Enter elements of B (1 x 1):

3

Enter elements of C (1 x 1):

4

LHS = $A*(B+C)$:

14

RHS = $(A*B) + (A*C)$:

14

Left distributive property is VERIFIED.

Left distributive property is VERIFIED.

$$A*(B+C) = (A*B) + (A*C)$$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property

$$A*(B+C) = (A*B) + (A*C)$$

2. Verify Right Distributive Property

$$(A+B)*C = (A*C) + (B*C)$$

3. Exit

Enter your choice: 5

Invalid choice! Please enter 1, 2, or 3.

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property

$$A*(B+C) = (A*B) + (A*C)$$

2. Verify Right Distributive Property

$$(A+B)*C = (A*C) + (B*C)$$

3. Exit

Enter your choice: 1

— Left Distributive Property —

$$A*(B+C) = (A*B) + (A*C)$$

Enter m, n and p: 2 3 2

Enter elements of A (2 x 3):

1 2 3 4

5

6

Enter elements of B (3 x 2):

5 4 3 2

1 6 7 8

Enter elements of C (3 x 2):

6 7 8 9

LHS = A*(B+C):

57 75

147 183

RHS = (A*B) + (A*C):

57 75

147 183

Left distributive property is VERIFIED.

$A*(B+C) = (A*B) + (A*C)$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property

$A*(B+C) = (A*B) + (A*C)$

2. Verify Right Distributive Property

$(A+B)*C = (A*C) + (B*C)$

3. Exit

Enter your choice: 2

— Right Distributive Property —

$(A+B)*C = (A*C) + (B*C)$

Enter m, n and p: 2 2 2

Enter elements of A (2 x 2):

1 2 3 4

Enter elements of B (2 x 2):

4 3 2 1

Enter elements of C (2 x 2):

5 6 7 8

LHS = (A+B)*C:

60 70

60 70

RHS = (A*C) + (B*C):

60 70

60 70

Right distributive property is VERIFIED.

$(A+B)*C = (A*C) + (B*C)$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property
 $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property
 $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice: 2

— Right Distributive Property —

$$(A+B)*C = (A*C) + (B*C)$$

Enter m, n and p: 2 3 1

Enter elements of A (2 x 3):

1 2 3 4 5 6 7 8

Enter elements of B (2 x 3):

8 7 6 5 4 3 2 1

Enter elements of C (3 x 1):

LHS = $(A+B)*C$:

84

99

RHS = $(A*C) + (B*C)$:

84

99

Right distributive property is VERIFIED.

$$(A+B)*C = (A*C) + (B*C)$$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property
 $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property
 $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice:

— Left Distributive Property —

$$A*(B+C) = (A*B) + (A*C)$$

Enter m, n and p: 1

0 2 2

Dimensions must be positive. Enter again.

Enter m, n and p: -2 2 3

Dimensions must be positive. Enter again.

Enter m, n and p: 1 1 1

Enter elements of A (3 x 1):

6

3

Enter elements of B (1 x 1):

8

Enter elements of C (1 x 1):

3

LHS = $A*(B+C)$:

11

66

33

RHS = $(A*B) + (A*C)$:

11

66

33

Left distributive property is VERIFIED.

$$A*(B+C) = (A*B) + (A*C)$$

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property

$$A*(B+C) = (A*B) + (A*C)$$

2. Verify Right Distributive Property

$$(A+B)*C = (A*C) + (B*C)$$

3. Exit

Enter your choice: 4

Enter your choice: 4
Invalid choice! Please enter 1, 2, or 3.

===== MATRIX DISTRIBUTIVE PROPERTIES =====

1. Verify Left Distributive Property
 $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property
 $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice: 3
Exiting the program...

(kali@kali)-[~/Desktop/exp34]
\$